

Business Requirements Document (BRD)

Project Name: Remote Internship Progress Dashboard

Document Version: 1.0

Status: Draft

Prepared By: Utkarsh Dwivedi

Date: 09 April 2026

1. Executive Summary

1.1. Project Vision The "Remote Internship Progress Dashboard" is a centralized web-based internal portal designed to bridge the gap between mentors and remote interns. It replaces fragmented communication channels (like emails or scattered spreadsheets) with a unified system.

1.2. Problem Statement Currently, tracking the daily/weekly progress of remote interns is challenging. Mentors struggle to monitor task statuses in real-time, and interns lack a standardized format to report their learnings and roadblocks. This leads to delayed feedback, miscommunication, and difficulty in evaluating an intern's overall performance.

2. Business Objectives & ROI

- **Centralized Task Management:** Achieve a 100% transparent workflow using a Kanban methodology.
- **Standardized Evaluation:** Implement Markdown-based weekly reports to ensure consistency in student submissions.
- **Data-Driven Mentorship:** Utilize Python-powered backend analytics to generate performance metrics (e.g., task completion rate, average time per task), allowing mentors to identify struggling interns early.
- **Time Efficiency:** Reduce the time mentors spend manually tracking down updates by at least 40%.

3. User Roles & Personas

Role	Access Level	Key Activities & Permissions

Super Admin / Program Manager	Full Access	Can add/remove mentors and interns, assign mentors to interns, view global analytics for the entire internship cohort.
Mentor	Elevated Access	Can create, assign, and update tasks. Can view reports of their assigned interns, leave feedback, and view individual student analytics.
Intern (Student)	User Access	Can view assigned tasks, update task status (move Kanban cards), write/edit/submit Markdown reports, and view mentor feedback.

4. Functional Requirements (In-Depth)

4.1. Authentication & Authorization

- **FR-1.1:** The system must support Secure Login (Email/Password) with JWT (JSON Web Tokens) based session management.
- **FR-1.2:** Role-based redirection upon login (Mentors go to the Mentor Dashboard; Interns go to the Intern Kanban View).

4.2. Task Management Module (Kanban Board)

- **FR-2.1:** The UI must implement `react-kanban` for an interactive, drag-and-drop board interface.
- **FR-2.2:** Default columns: *Backlog*, *To Do*, *In Progress*, *Blocker/Issue*, *Review*, *Done*.
- **FR-2.3:** Task Card Details: Each card must contain a Title, Description, Assigned Date, Deadline, Priority Tags (High, Medium, Low), and Status.
- **FR-2.4:** Mentors can create and assign cards. Interns can only change the status (move columns) or add comments to the card.

4.3. Markdown-Based Reporting Module

- **FR-3.1:** The system must feature a Markdown editor (with live preview) for drafting weekly reports.
- **FR-3.2:** Report templates should be pre-loaded (e.g., "Tasks Completed", "Learnings", "Roadblocks", "Plan for Next Week").
- **FR-3.3: Autosave Feature:** Drafts must be automatically saved locally or to the database to prevent data loss.
- **FR-3.4:** Once submitted, the report becomes read-only for the intern, and the mentor receives a notification.

4.4. Mentor Feedback & Communication

- **FR-4.1:** Mentors must be able to view rendered Markdown reports and leave inline comments or a general "Review Summary".
- **FR-4.2:** Mentors can flag a report as "Needs Revision," which unlocks the report for the intern to edit and resubmit.

4.5. Python Progress Analytics Engine

- **FR-5.1:** The Python backend must run scheduled jobs (e.g., using Celery or CRON) to calculate weekly performance metrics.
- **FR-5.2:** Data points to calculate:
 - *Task Velocity:* Number of tasks moved to "Done" per week.
 - *Bottleneck Analysis:* Average time a task spends in the "In Progress" or "Blocker" column.
 - *Submission Consistency:* Timeliness of weekly report submissions.
- **FR-5.3:** Frontend visualization of these analytics using charts (e.g., Bar charts for weekly tasks, Pie charts for task priority distribution).

5. Technical Requirements & Architecture

5.1. Tech Stack Overview

- **Frontend Development:** React.js, Context API/Redux (for state management), [react-kanban](#) (for board UI), [react-markdown](#) (for report rendering), TailwindCSS/Material-UI (for styling).
- **Backend Development:** Python with Django REST Framework (DRF) or FastAPI. Python's [pandas](#) library can be utilized for processing complex analytics data.
- **Database:** PostgreSQL (Relational structure is best for linking Users, Tasks, and Reports).

5.2. Data Models (High-Level)

- **User Table:** UserID, Name, Email, Role (Mentor/Intern), CreatedAt.
- **Task Table:** TaskID, AssignedTo, CreatedBy, Title, Description, Status, DueDate.
- **Report Table:** ReportID, InternID, WeekNumber, Content (Markdown String), Status (Draft/Submitted/Reviewed), MentorFeedback.

6. Non-Functional Requirements (NFRs)

- **Performance:** The Kanban board must render and update within 300ms to ensure a smooth drag-and-drop experience. Complex analytics API queries must resolve in under 2 seconds.
- **Scalability:** The architecture must support up to 500 concurrent users (interns + mentors) without performance degradation.

- **Security:** Password hashing (e.g., bcrypt). API endpoints must be secured against SQL injection and Cross-Site Scripting (XSS), especially since users are inputting Markdown text.
- **Usability:** The interface must be responsive, working seamlessly on both desktop monitors and laptop screens (mobile view is secondary for this internal tool).

7. Assumptions, Dependencies & Risks

- **Assumptions:** All interns have a basic understanding of Markdown syntax. The organization will provide the hosting infrastructure (AWS/GCP/Heroku).
- **Dependencies:** The frontend heavily relies on the [react-kanban](#) library being actively maintained and compatible with the chosen React version.
- **Risks:** If interns fail to move Kanban cards or submit reports on time, the Python analytics engine will display inaccurate/skewed data. *Mitigation:* Implement automated email/dashboard reminders for pending tasks and reports.